

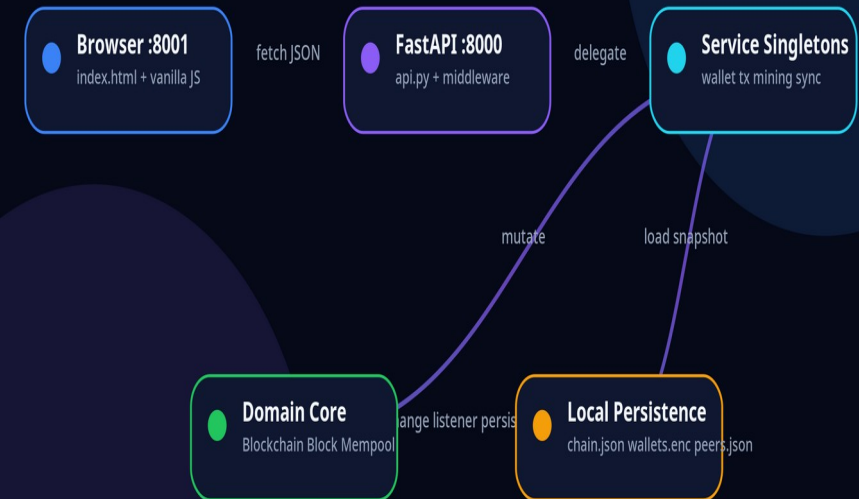
CHAIN EXPLORER

A source-grounded blockchain systems presentation

Educational account-model PoW simulator · FastAPI · Vanilla JS · SHA-256
JSON/Fernet persistence

Real Runtime Architecture

What actually runs locally: browser static frontend, FastAPI process, JSON/Fernet state.



July 02, 2026

PROBLEM

Blockchain education hides the implementation behind abstractions

Learners see terms

Hash, nonce, mempool, signature, fork and consensus are usually introduced as vocabulary before students see state changes.

The code path is invisible

A UI click often hides request validation, signing, balance checks, mining, persistence and chart aggregation.

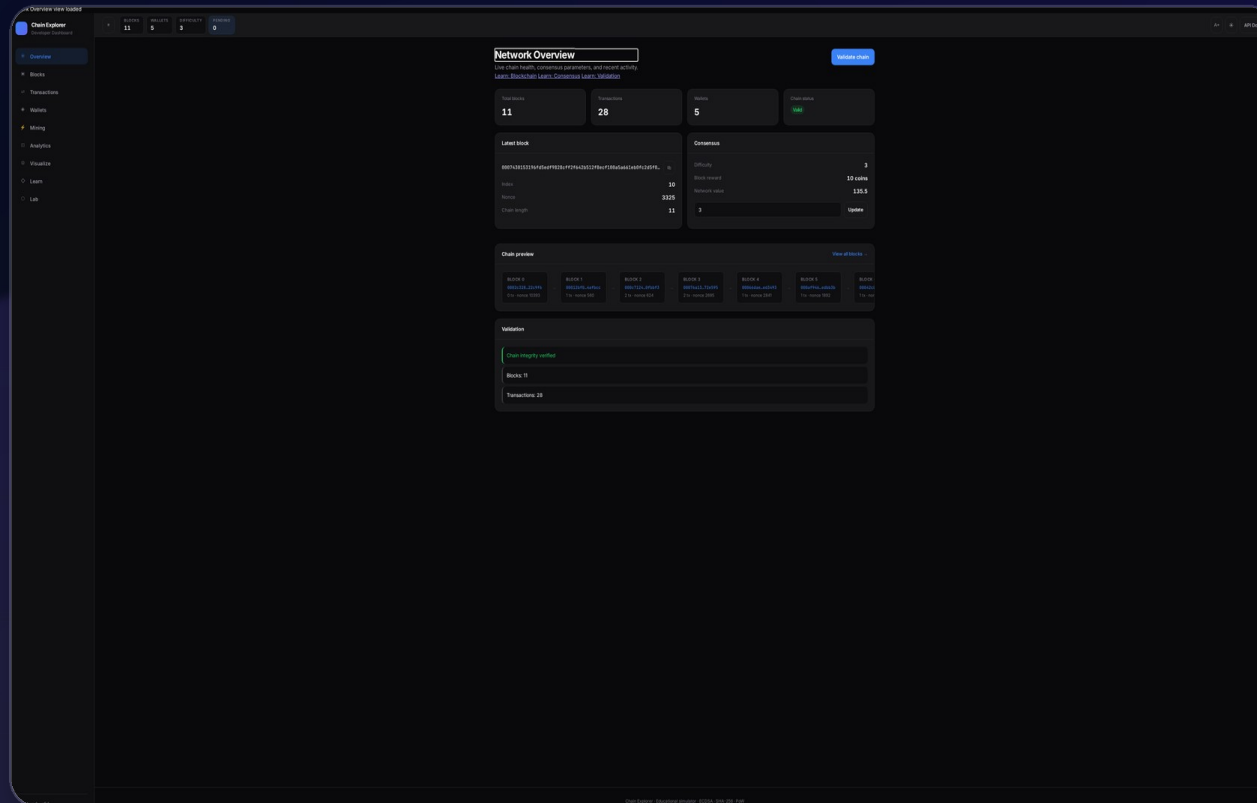
Production chains are too large

Bitcoin/Ethereum are accurate, but too complex for first-principles architecture review in one defense.

This project solves the observability problem: it makes the ledger, API, validation rules and security failures inspectable in one local system.

SOLUTION

A teaching ledger where every concept maps to running code



Local-first

Two processes: static frontend and FastAPI API.

Inspectable

OpenAPI docs, live stats, blocks, mempool, wallets.

Safe experiments

Tamper and 51% demos are educational and isolated.

The presentation is derived from the implementation, not generic blockchain copy

Backend

36 Python modules read: routes, services, domain, persistence, sync, middleware.

Frontend

32 HTML/CSS/JS files read: view shell, rendering, cache, lazy modules, a11y.

Runtime

23 OpenAPI paths discovered; live chain validates with 11 blocks.

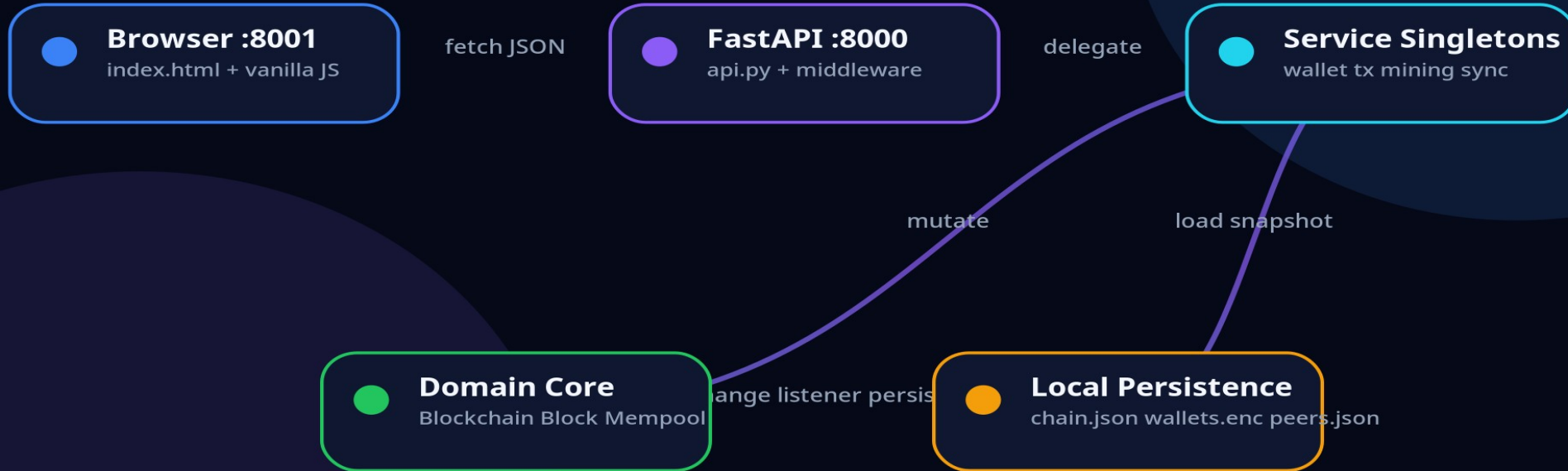
Not detected

No auth, SQL database, Docker, workers, React build, or admin panel in source.

The actual deployment is intentionally small

Real Runtime Architecture

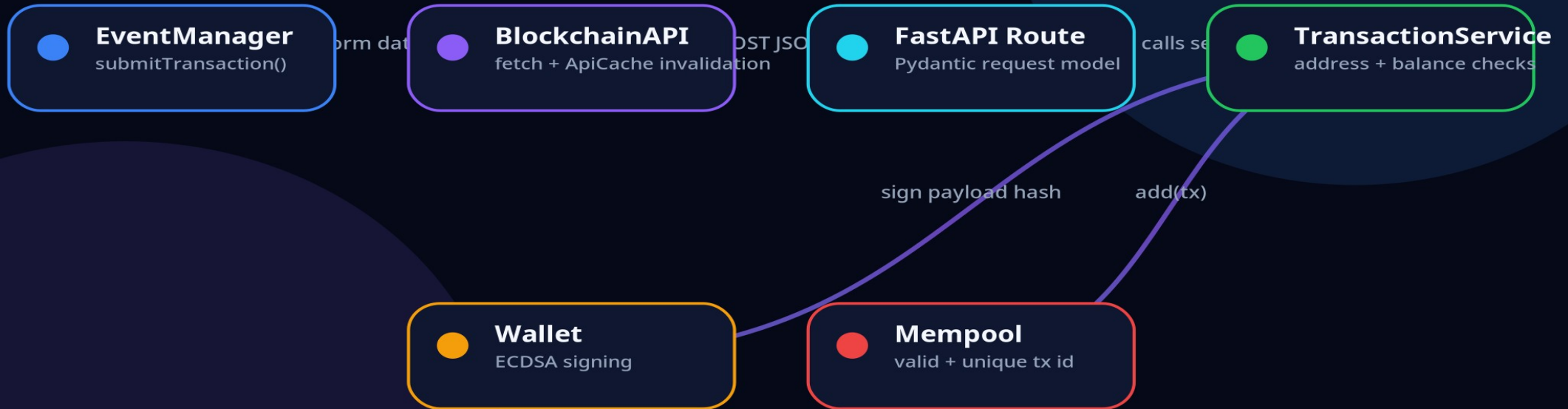
What actually runs locally: browser static frontend, FastAPI process, JSON/Fernet state.



A transfer is admitted only after route validation, signing, balance reservation and mempool checks

Real Request Lifecycle

POST /transaction/create from form submission to mempool admission.



api.py stays thin because behavior lives in dedicated orchestration services

BlockchainService

Loads/saves chain, validates chain, exposes stats, replaces longer valid chains.

WalletService

Creates ECDSA wallets, keeps names, persists private keys through WalletStore.

TransactionService

Validates addresses/amounts, checks pending outgoing balance, signs, admits to mempool.

MiningService

Validates miner wallet, creates Miner, mines synchronously, returns nonce/time/hash.

AnalyticsService

Computes chart series, miner rewards, storage size, health and sync progress.

SyncService

Registers peers and asks ChainSynchronizer for longer chains.

The simulator chooses account balances because the learning objective is transaction flow

Implemented

Account balances are not stored as rows. `Blockchain.get_balance()` rebuilds a cache by replaying confirmed transactions and invalidates it on mutation.

Why here

For teaching, Alice -> Bob transfers are easier to follow than UTXO coin selection. The trade-off is less production realism.

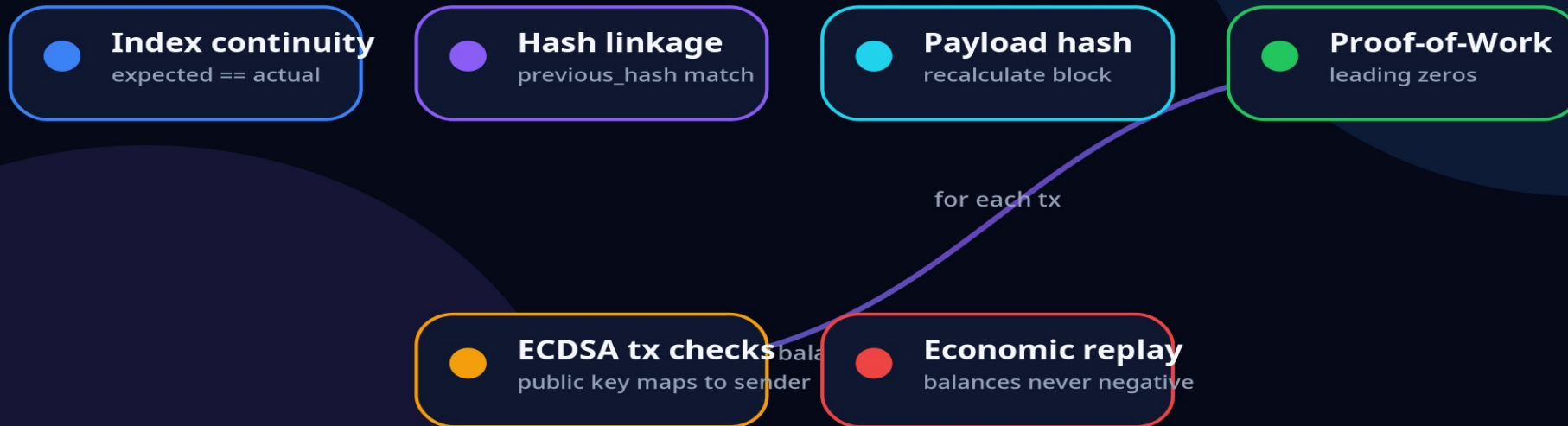
Limits

Float amounts, no account nonce, no fees, no gas, no supply cap, no Merkle root in block headers.

Consensus safety is enforced by deterministic replay

Real Chain Validation Pipeline

The exact checks inside `Blockchain.is_chain_valid()`.



Real primitives are used, but custody is simplified for education

SHA-256

Blocks and transactions hash deterministic JSON using ``sha256_dict(..., legacy=True)`` so stored chains remain compatible.

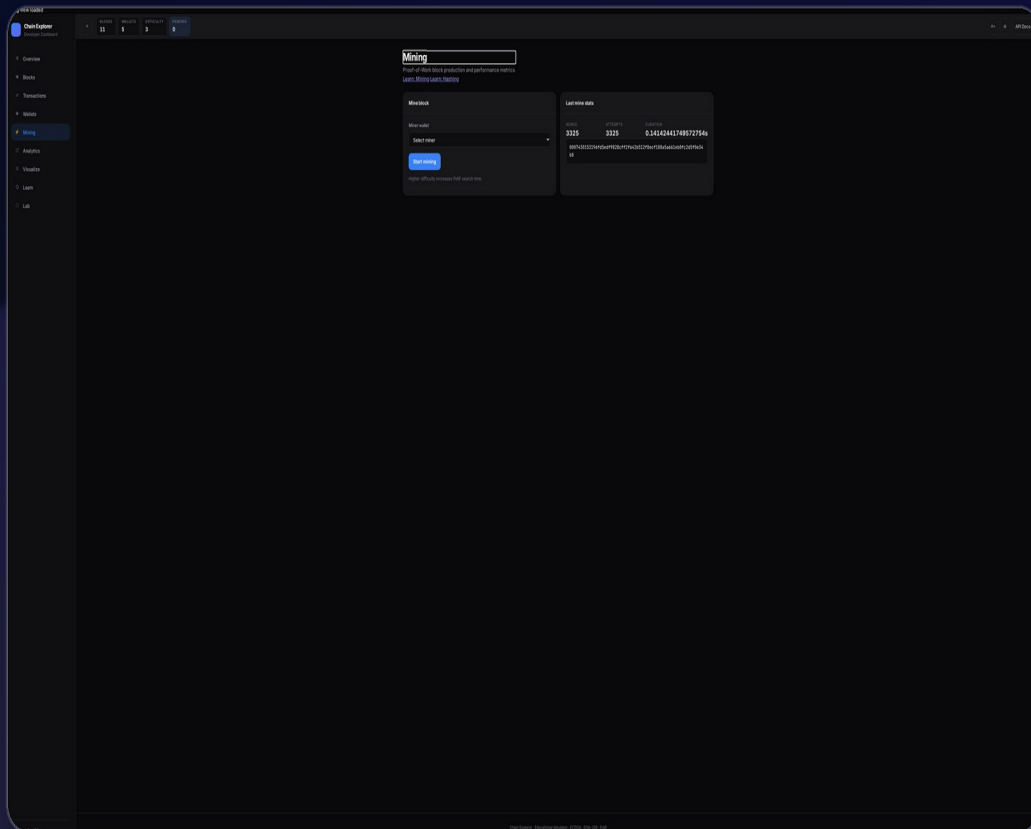
ECDSA NIST256p

Wallets generate ECDSA key pairs. Transactions sign the payload hash and validation checks public key -> address mapping.

Fernet at rest

Private keys are stored in ``data/wallets.enc``; the API never returns private keys, but signing is server-side by design.

Proof-of-Work is implemented as a visible nonce search



Target

``hash.startswith('0' * difficulty)``

Reward

``SYSTEM`` transaction mints 10.0 coins

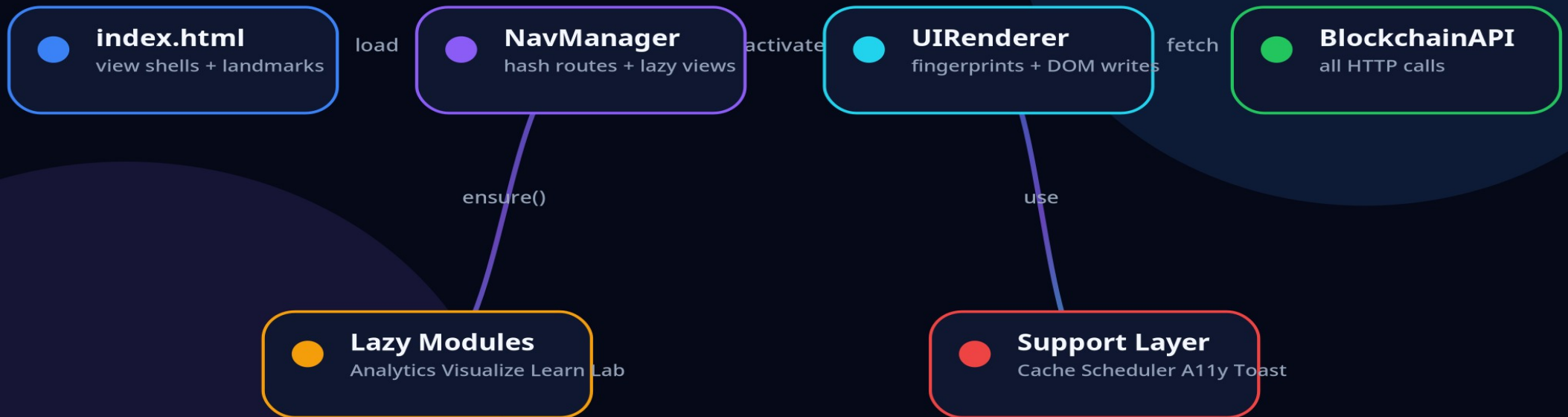
Trade-off

Mining runs synchronously on the API thread. This is acceptable at low difficulty and makes the educational timing visible.

The browser architecture is intentionally transparent

Real Frontend Architecture

No React build: script-loaded modules, hash navigation, lazy feature bundles.



Most optimizations are small, local, and visible in source

ApiCache

TTL cache plus in-flight dedup prevents duplicate GET calls.

Mutation invalidation

POST actions invalidate affected prefixes: blocks, stats, wallets, analytics.

Fingerprints

Renderer skips DOM updates when block/stat/wallet fingerprints are unchanged.

VirtualList

Blocks table windows rows after 40 items.

Scheduler

DOM writes are batched with requestAnimationFrame; polling pauses when tab is hidden.

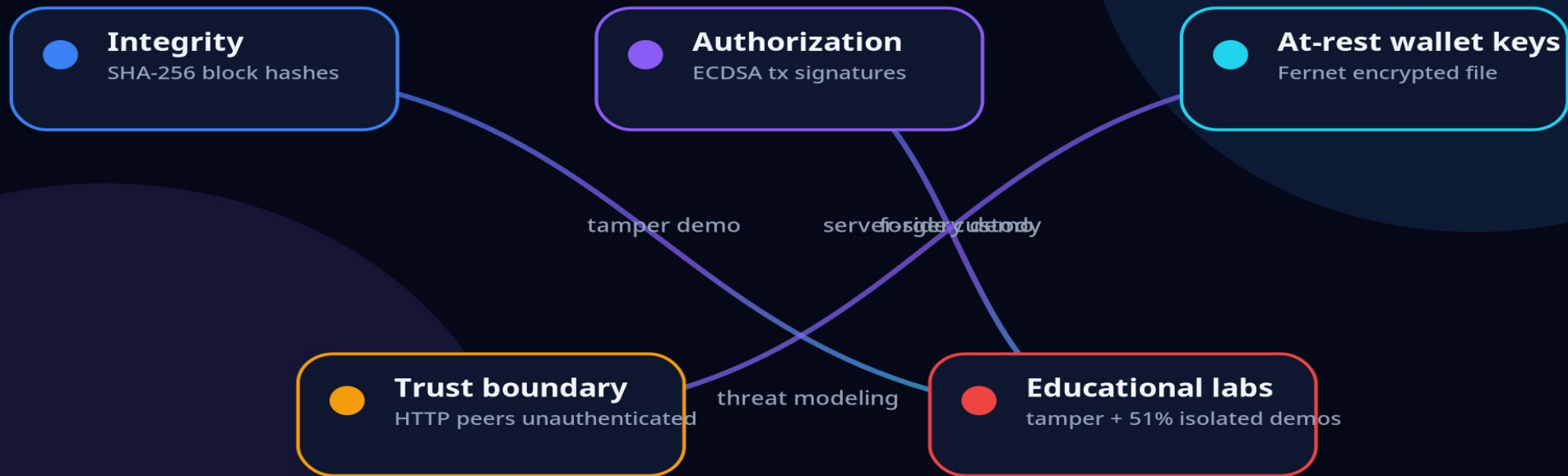
Lazy modules

Chart.js, Visualize, Learn and Lab load only when opened.

The security model is honest about its trust boundaries

Real Security Architecture

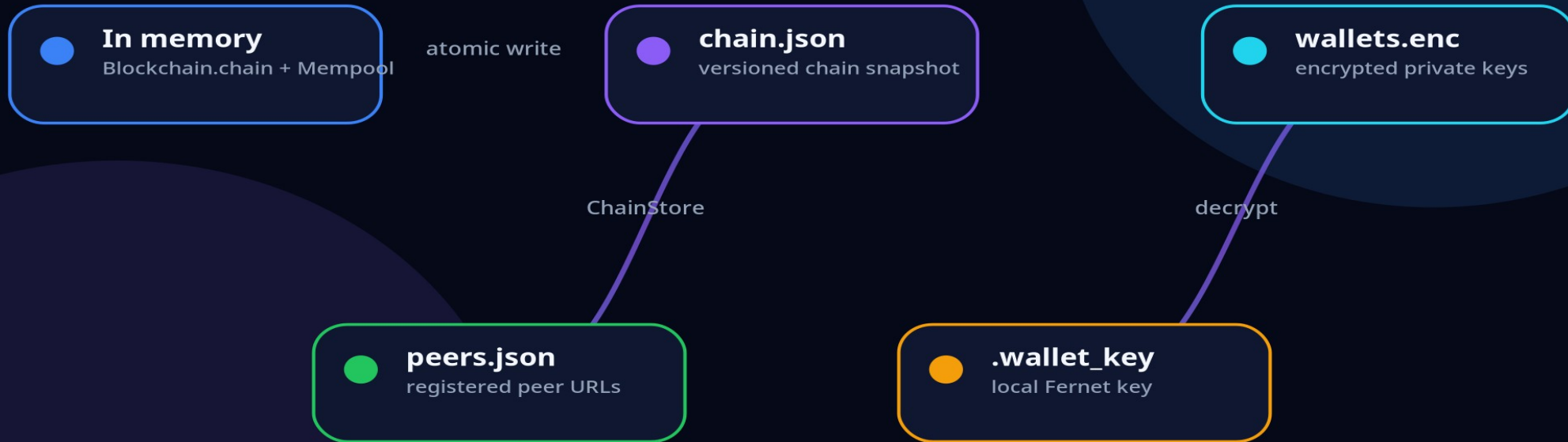
What is protected, what is intentionally simplified, and where the trust boundary is.



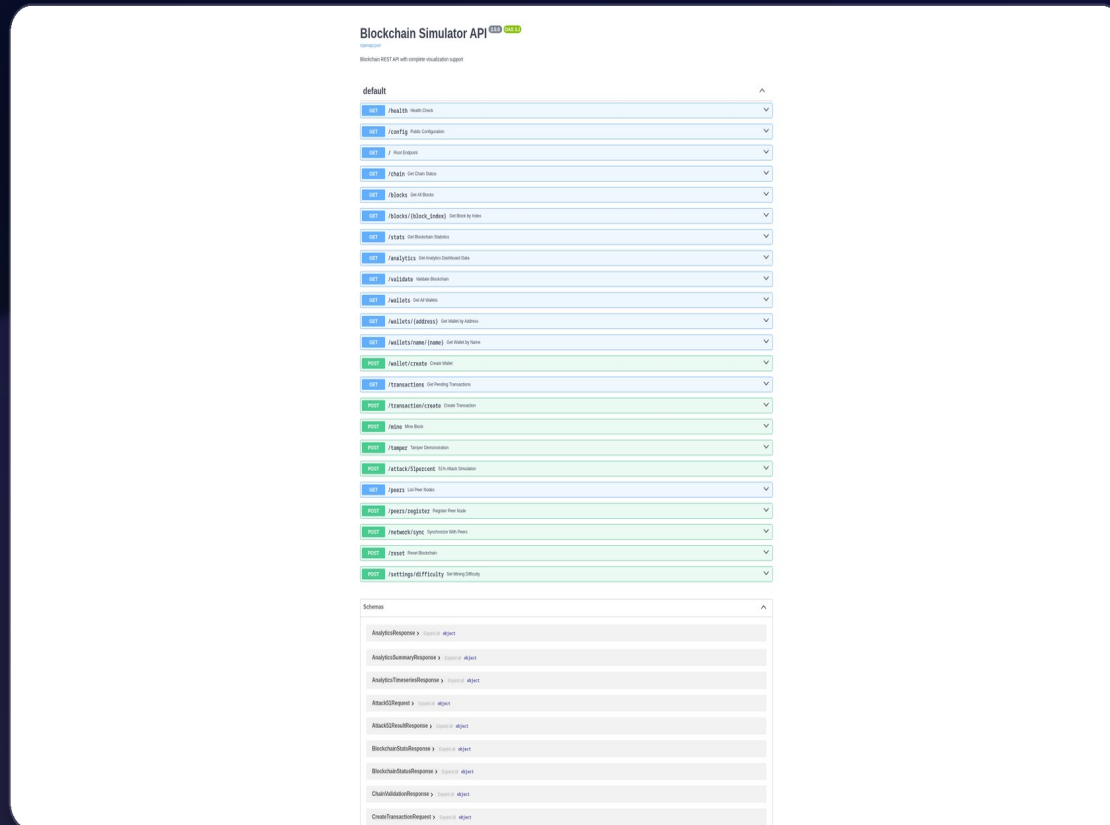
There is no hidden database: persistence is JSON and encrypted files

Real Data Architecture

There is no database server: state is serialized domain objects on disk.



The REST surface mirrors the teaching workflow



Observe

/health /chain /stats /analytics /blocks

Act

/wallet/create /transaction/create /mine

Stress

/tamper /attack/51percent /validate

Network

/peers /peers/register /network/sync

The charts are computed from chain objects, not a

Height

10

Tx/block

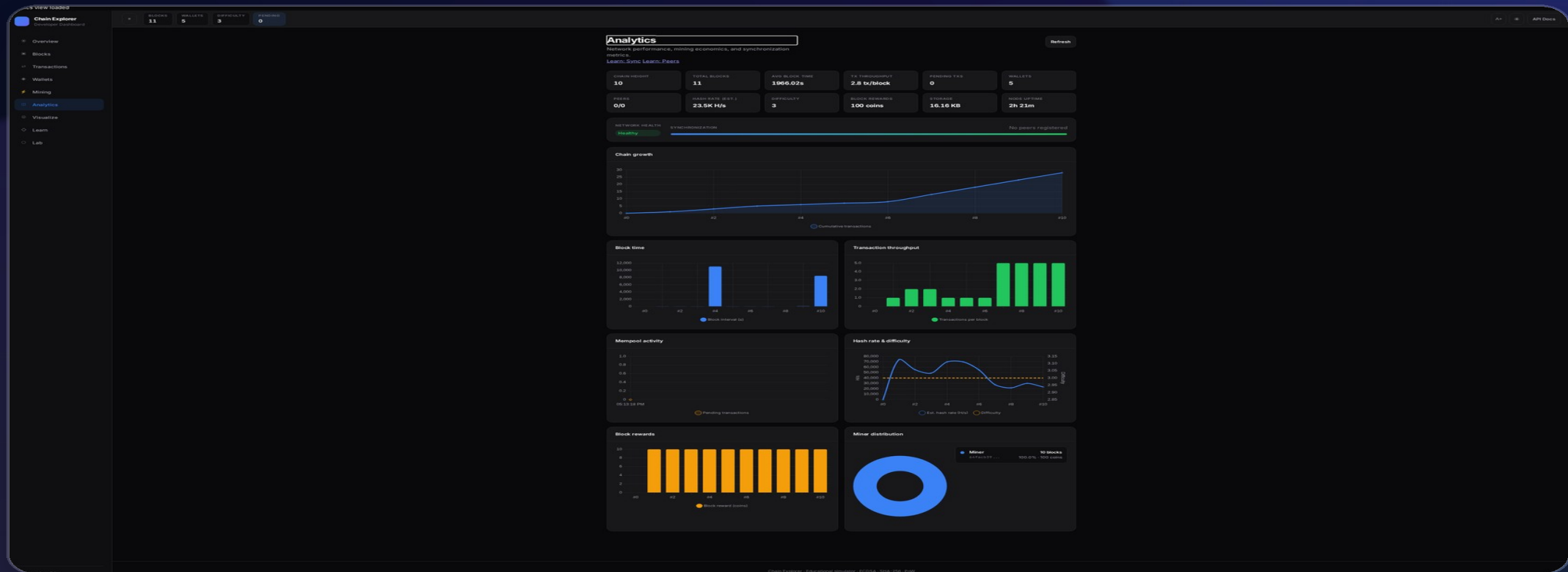
2.8

Hash rate

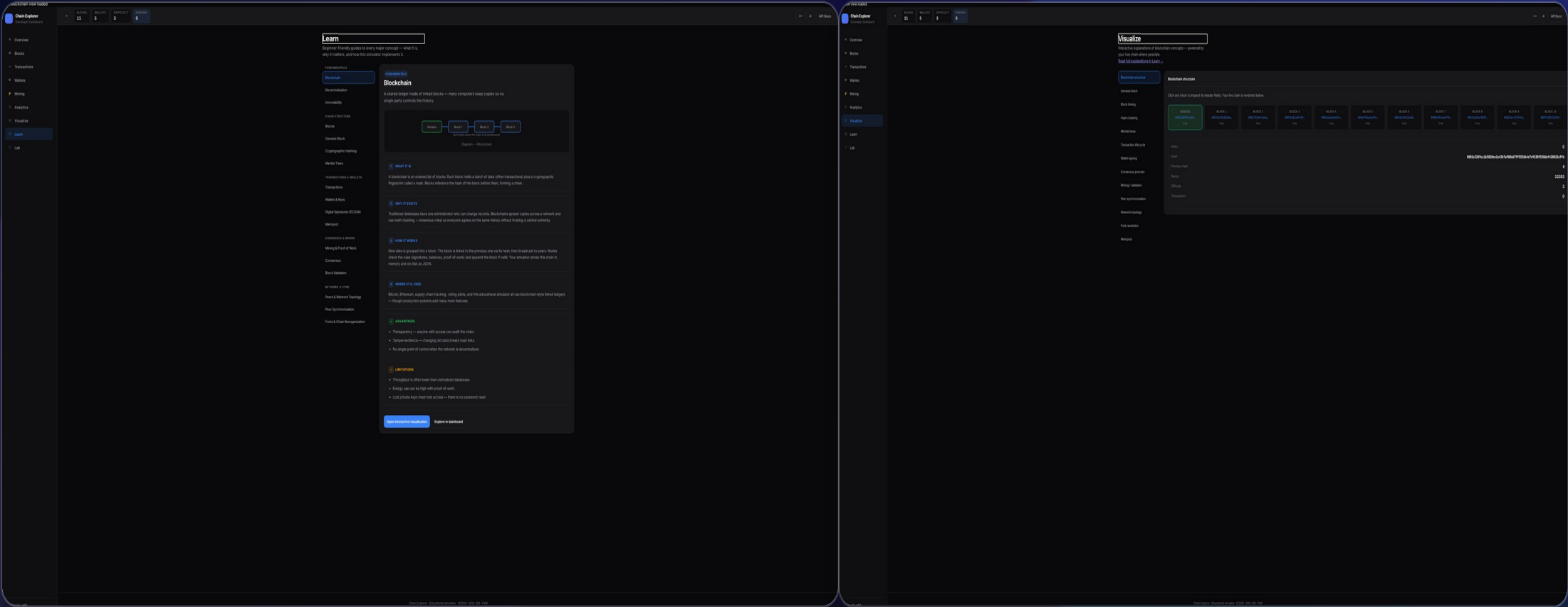
23.5K H/s

Storage

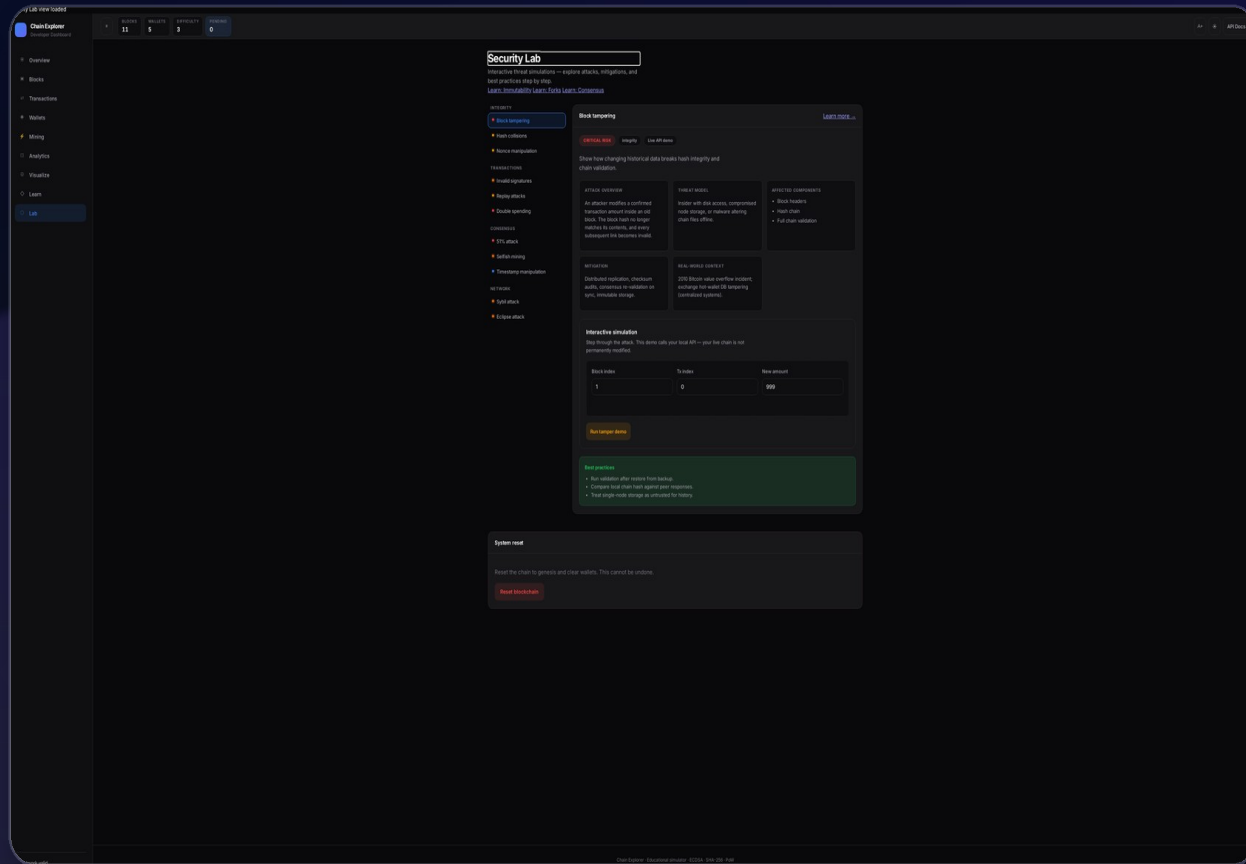
16.16 KB



Education content is wired back to live visualizations and dashboard views



Threat modeling is part of the product, not an appendix



Threat catalog

11 topics across integrity, transaction, consensus and network categories.

Live API demos

Tamper detection and 51% attack call backend endpoints.

Non-destructive

Tamper service restores original chain state after demonstration.

The dashboard is built for classroom and defense environments

Landmarks

main, banner, nav, contentinfo and table captions.

Keyboard

Tab order, Escape for mobile menu, '?' shortcuts, roving topic lists.

Announcements

ARIA live region for view changes, status and connection events.

Contrast

A+ toggle, prefers-contrast and forced-colors rules.

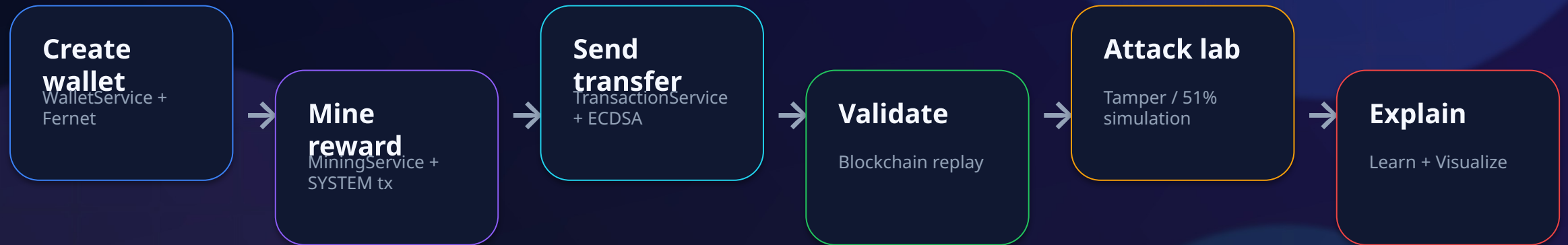
Motion

prefers-reduced-motion disables animation-heavy effects.

Touch

44px targets for buttons and navigation.

A learner moves from identity to validated history



Every simplification serves the educational simulator

FastAPI

Chosen here because Pydantic route models and OpenAPI make API behavior inspectable during demos.

Vanilla JS

Chosen here so students can read the UI without a bundler, component compiler or framework layer.

JSON persistence

Chosen here because chain snapshots can be opened and compared during architecture review.

Server-side signing

Chosen here to keep wallet UX simple; documented as non-production custody.

HTTP sync

Chosen here to demonstrate longest-chain adoption without implementing native P2P gossip.

Account balances

Chosen here to teach transfer flow before UTXO complexity.

The project is clear about what it does not implement

No authenticated peers

HTTP sync trusts registered URLs.

No transaction nonce

Mempool dedup prevents exact duplicates, not full account replay semantics.

Float amounts

Acceptable for demos, not financial precision.

No Merkle header

Visualize teaches Merkle trees; block hash uses flat tx list.

Synchronous mining

Simple for low difficulty; unsuitable for heavy workloads.

No production wallet custody

Private keys are server-side for teaching UX.

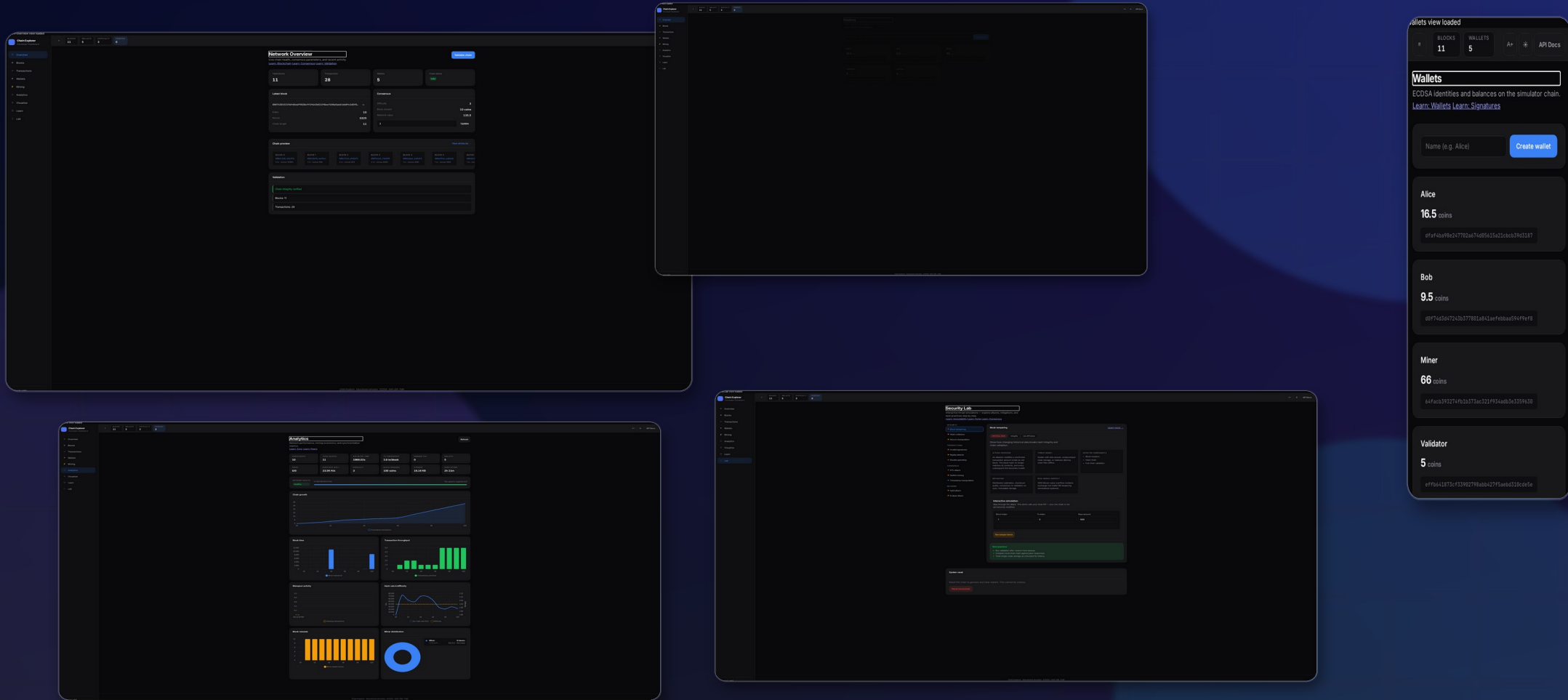
Chain Explorer is a microscope, not a public chain

Implementation-aligned matrix

Dimension	Chain Explorer	Bitcoin	Ethereum
Purpose	Education	Peer money	Programmable settlement
Ledger	Account balances	UTXO	Accounts + contracts
Consensus	PoW demo	PoW	PoS
Network	HTTP sync	Native P2P	P2P
Storage	JSON/Fernet	Client DB	Client DB
Contracts	Not implemented	Script	EVM

LIVE DEMO

The system can be presented as a coherent product



The next work should preserve explainability while adding guardrails

Tests

pytest for validation, crypto, services; UI smoke via Playwright.

Docker

Repeatable multi-node classroom labs.

Client keys

Move signing to browser for production-like custody.

Auth for peers

Signed peer metadata or allowlist trust model.

Integers

Replace float amounts with smallest-unit integers.

CI/CD

Lint, tests, artifact build and presentation regeneration.

Conclusion

A complete learning platform for blockchain systems architecture

Technical contribution

Connects cryptography, validation, mining, persistence, APIs and UI state in one inspectable simulator.

Educational contribution

Turns hidden blockchain mechanics into interactive workflows and attack simulations.

Engineering maturity

Explicit boundaries, documented trade-offs, caching, a11y, structured errors and reproducible artifacts.

Thank you. Questions?